

Guião de Simulação de Perguntas e Respostas com Júri de Defesa

Projeto PlataformaGestaoHorarios — Plataforma de Gestão de Horários (Levi's Store)

Tiago

Francisco

20 de junho de 2026

Resumo

Este documento constitui um guião de preparação para a defesa pública do projeto *PlataformaGestaoHorarios*, um sistema híbrido desktop (JavaFX) e web (Spring MVC + Thymeleaf) que partilha uma camada de lógica de negócio comum sobre uma base de dados PostgreSQL. As perguntas e respostas aqui simuladas resultam de uma análise exaustiva e atual do código-fonte do repositório, cobrindo arquitetura, mapeamento ecrã-backend, auditoria de segurança (OWASP) e evolução de schema de base de dados.

Conteúdo

1	Introdução à Arquitetura	2
2	O Dicionário de Ecrãs	2
2.1	Ecrã de Login	3
2.2	Dashboard de Horários	3
2.3	Gestão de Equipa / Turnos	4
2.4	Preferências, Folgas e Permutas (Complementares)	4
3	Auditoria OWASP e Segurança	5
4	Evolução e Schema de Base de Dados	6

1 Introdução à Arquitetura

Pergunta do Professor: Vocês têm uma aplicação desktop em JavaFX e uma aplicação web em Spring MVC. Isso obriga-vos a duplicar toda a lógica de negócio duas vezes, uma para cada interface?

Resposta do Aluno (Tiago/Francisco):

Não. A nossa arquitetura assenta num único projeto Maven dividido em três módulos: API/, DESKTOP/ e WEB/. O módulo API/ contém toda a lógica de negócio — as entidades JPA em Modules/, os repositórios Spring Data em Repositories/ e, sobretudo, os serviços em Services/, onde reside a totalidade das regras de negócio. Tanto os controllers do DESKTOP/ como os do WEB/ injetam exatamente as mesmas classes @Service. Isto significa que uma regra como a validação de descanso mínimo entre turnos, implementada uma única vez em HorarioValidatorService, é automaticamente respeitada quer o pedido venha do ecrã desktop quer venha do portal web.

Pergunta do Professor: Mas então como é que o JavaFX, que corre num processo desktop, acede aos mesmos serviços Spring que o servidor web expõe via HTTP? Isso implica uma API REST interna?

Resposta do Aluno (Tiago/Francisco):

Não existe nenhuma fronteira REST entre o desktop e a base de dados. O ponto de entrada do desktop, a classe AppLauncher, arranca o próprio contexto Spring com WebApplicationType.NONE — ou seja, sobe um contexto de aplicação Spring completo, com todos os @Service e @Repository disponíveis para injeção, mas sem nunca iniciar um servidor HTTP. Os controllers JavaFX, como o DashboardController, recebem os serviços diretamente por injeção de dependências do Spring, tal como um controller MVC receberia. Chamam métodos Java normais, dentro do mesmo processo e da mesma JVM, sem serialização, sem rede e sem latência de rede. O módulo WEB/, por sua vez, arranca um contexto Spring Boot completo com servidor embutido (porta 8081) através de Projeto2WebApplication, expondo Thymeleaf para HTML e alguns @RestController só para chamadas AJAX internas do próprio browser — nunca para comunicação entre desktop e servidor.

Pergunta do Professor: Qual é a vantagem real desta decisão de arquitetura, comparada com teres duas aplicações totalmente independentes com a sua própria cópia da lógica de negócio?

Resposta do Aluno (Tiago/Francisco):

A vantagem principal é a garantia de consistência de comportamento. Se corrigirmos um bug de regra de negócio — por exemplo, na geração de horários, dentro do motor HorarioGeneratorEngine em API/Services/geracao/ — essa correção propaga-se automaticamente a ambas as interfaces, porque ambas chamam a mesma instância de serviço gerida pelo Spring. Isto elimina o risco clássico de *drift* entre duas implementações duplicadas, reduz a superfície de testes (testamos a lógica de negócio uma única vez, em src/test/java, e essa cobertura vale para as duas interfaces) e simplifica a manutenção a longo prazo de um projeto académico com prazo apertado.

2 O Dicionário de Ecrãs

Nesta secção respondemos, ecrã a ecrã, à pergunta típica de júri: “Se eu clicar aqui, o que acontece no código?”

2.1 Ecrã de Login

Pergunta do Professor: Eu cliço no botão de login no vosso portal web. Mostrem-me exactamente onde está esse HTML, qual o Controller que recebe o pedido, e onde é validada a password.

Resposta do Aluno (Tiago/Francisco):

Vista HTML: `src/main/resources/templates/web/login.html`.

Controller: `WEB/WebLoginController.java`, anotado com `@RequestMapping("/web")`.

Rota GET (mostrar página): `GET /web/login`, método `loginPage(...)`.

Rota POST (submeter credenciais): `POST /web/login`, método `autenticar(...)`, que recebe email e password via `@RequestParam`.

Service: o controller delega de imediato em `UtilizadorService.efetuarLogin(email, password)`. Esse método, por sua vez, usa o `SegurancaService` para normalizar o email (`normalizarEmailLogin`) e para comparar a password fornecida com o hash guardado (`passwordCorresponde`).

Pós-login: se a autenticação for válida, o `WebLoginController` consulta `LojautilizadorRepository.findLig` para decidir se o utilizador tem acesso a uma única loja — e segue direto para `/web/horarios` — ou a múltiplas lojas, sendo nesse caso redirecionado para `GET /web/selecionar-loja`.

Pergunta do Professor: E a password, está guardada em texto simples na base de dados? Isso seria gravíssimo.

Resposta do Aluno (Tiago/Francisco):

Não. Toda a manipulação de passwords passa pelo `SegurancaService`, que centraliza o hashing (`gerarHash`), a verificação de correspondência (`passwordCorresponde`) e inclusive a deteção de passwords antigas ainda não migradas para hash (`precisaMigrarParaHash`), o que demonstra uma estratégia de migração progressiva de segurança sem obrigar a um *reset* em massa de credenciais de utilizadores existentes.

2.2 Dashboard de Horários

Pergunta do Professor: O ecrã de horários tem um calendário interativo. Isso implica reload de página a cada clique, ou têm algum mecanismo assíncrono?

Resposta do Aluno (Tiago/Francisco):

Vista HTML: `src/main/resources/templates/web/horarios.html`.

Controller da página: `WEB/WebHorariosController.java`, em `@RequestMapping("/web/horarios")`, com a rota `GET /web/horarios` a carregar a grelha mensal inicial e `GET /web/horarios/exportar.pdf` a gerar o PDF do horário do mês.

Controller AJAX: o calendário interativo, sem reload, é alimentado pelo `WEB/WebHorariosApiController.java` que expõe `GET /api/horarios/mes` e `GET /api/horarios/detalhe-dia` para o JavaScript do lado do cliente consultar dados pontuais sem recarregar a página inteira.

Service: ambos os controllers delegam em `GeracaoHorariosService.obterMeusHorarios(idUtilizador, idLoja, ano, mes)`, que por sua vez consulta o `HorarioRepository`.

Exportação PDF: a rota de exportação delega em `WebPdfService`, que reutiliza o mesmo `PdfExportService` (Apache PDFBox) usado pelo desktop — outra prova concreta da partilha de lógica entre módulos.

Pergunta do Professor: E o algoritmo que efetivamente gera o horário, onde é invocado e que tipo de algoritmo é?

Resposta do Aluno (Tiago/Francisco):

A geração de propostas é despoletada por `GeracaoHorariosService.gerarPropostas(...)`, um método `@Transactional` que prepara os dados de entrada (`prepararDadosGeracao`) e invoca o motor `HorarioGeneratorEngine`, localizado em `API/Services/geracao/`. Trata-se de um algoritmo de *backtracking* com restrições (*constraint solver*), apoiado por classes auxiliares especializadas como `PlaneadorFinsDeSemana`, `PlaneadorFolgasPreferidas`, `AvaliadorAtribuicao` e `PolisherHorario`, que aplicam, por esta ordem, a alocação inicial de turnos, a otimização de preferências e um refinamento final de equidade entre colaboradores.

2.3 Gestão de Equipa / Turnos

Pergunta do Professor: Como gerente de loja, eu preciso de aprovar pedidos de folga dos meus subordinados. Mostrem-me o caminho completo desse fluxo.

Resposta do Aluno (Tiago/Francisco):

Vista HTML: `src/main/resources/templates/web/equipa.html`.

Controller: `WEB/WebEquipaController.java`, em `@RequestMapping("/web/equipa")`.

Rotas de listagem e detalhe: `GET /web/equipa` (listagem geral) e `GET /web/equipa/{idUtilizador}` (detalhe individual de um colaborador).

Rota de aprovação: `POST /web/equipa/{idUtilizador}/folgas/{idDayOff}/aprovar` e o seu equivalente `/rejeitar`.

Service: a decisão é persistida através de `DayOffService`, que valida regras como o aviso mínimo de antecedência e se o mês já se encontra publicado antes de permitir a alteração de estado.

Rotas análogas: o mesmo controller expõe ainda `.../preferencias/{id}/aprovar|rejeitar` e `.../permutas/{id}/aprovar|rejeitar`, delegando respetivamente em `PreferenciaService` e `PermutaService`, e `.../reset-password` para reposição administrativa de credenciais.

2.4 Preferências, Folgas e Permutas (Complementares)

Pergunta do Professor: Um colaborador quer pedir uma troca de turno com um colega. Onde é que isso acontece e que validações de negócio existem para impedir abusos?

Resposta do Aluno (Tiago/Francisco):

Vista HTML: `src/main/resources/templates/web/complementares.html`.

Controller da página: `WEB/WebComplementaresController.java`, em `@RequestMapping("/web/complementares")` com a rota `GET /web/complementares` a carregar as listagens de folgas, preferências e permutas do próprio utilizador.

Rota de submissão de pedido de folga: POST /web/complementares/folgas.

Rota de submissão de preferência: POST /web/complementares/preferencias, restrita por validação a três tipos: folga_preferida, colegas e turnos.

Rota de submissão de permuta: POST /web/complementares/permutas, com aprovação em POST /web/complementares/permutas/{idPermuta}/aprovar.

APIs AJAX de permuta de turno: WEB/WebPermutasApiController.java, expondo GET /api/permutas para listar dinamicamente apenas os turnos compatíveis para troca.

APIs AJAX de permuta de folga: WEB/WebPermutaFolgaApiController.java, com POST /api/permuta-folga/submeter e POST /api/permuta-folga/cancelar.

Service e validações de negócio: PermutaFolgaService e PermutaService impõem, entre outras, a regra de descanso mínimo de 11 horas entre turnos trocados, a restrição de a troca ocorrer dentro do mesmo mês de calendário, e uma guarda de *cross-store* que impede a troca de turnos entre colaboradores associados a lojas diferentes.

Histórico do colaborador: PreferenciaService.listarColegasDaLoja(idUtilizador, idLoja) garante ainda que as listas de colegas disponíveis para preferências de equipa respeitam o âmbito (*scope*) da loja atual, e não toda a organização.

3 Auditoria OWASP e Segurança

Pergunta do Professor: Vocês afirmam ter implementado controlo de acesso. Mas eu, como qualquer utilizador autenticado de nível mais baixo, consigo simplesmente escrever /web/gestao-loja na barra de endereços e aceder a um módulo administrativo? Expliquem-me, com sinceridade, se isso já aconteceu no vosso código.

Resposta do Aluno (Tiago/Francisco):

Sim, e assumimos isso de forma transparente: foi um bug real, identificado e corrigido no commit `ceeac2f`, classificado como *Broken Access Control*, categoria A01:2021 do OWASP Top 10. A causa raiz não foi ausência de lógica de autorização — a lógica sempre existiu — mas sim um *drift* entre essa lógica e uma renomeação de rotas ocorrida num *pull request* anterior (PR#111). As rotas administrativas foram migradas de /web/modulos/gestao-loja para /web/gestao-loja, e de forma equivalente para relatorios, mas o WebGuardInterceptor não foi atualizado em conjunto, ficando a comparar contra o prefixo antigo, que deixou de existir em qualquer rota real.

Pergunta do Professor: Mostrem-me o código exato. Onde estava o erro, linha a linha?

Resposta do Aluno (Tiago/Francisco):

O WebGuardInterceptor regista-se em WebMvcConfig sobre o padrão `addPathPatterns("/web/**")`, com exclusão apenas de /web, /web/login, /web/logout e recursos estáticos. Dentro do método `preHandle`, depois de confirmar que existe sessão autenticada, era chamado o método privado `podeAcederAoModulo(path, idUtilizador, idLoja)`. Antes da correção, esse método continha:

```
if (path.startsWith("/web/modulos/gestao-loja")) {
    return webAppService.obterPermissoes(idUtilizador, idLoja)
        .podeGerirLoja();
}
if (path.startsWith("/web/modulos/relatorios")) {
```

```
        return webAppService.obterPermissoes(idUtilizador, idLoja)
            .podeVerRelatorios();
    }
    return true;
```

Como nenhuma rota real começava por `/web/modulos/...` após a migração, ambas as condições `startsWith` avaliavam sempre para `false`, e a execução caía diretamente no `return true` final — ou seja, qualquer utilizador autenticado, independentemente do seu cargo, era autorizado a aceder a `/web/gestao-loja` e `/web/relatorios` apenas por possuir sessão válida.

Pergunta do Professor: Qual foi o impacto real deste bug se tivesse chegado a produção? Não me dêem uma resposta vaga, quantifiquem o risco.

Resposta do Aluno (Tiago/Francisco):

O impacto real seria a exposição de funcionalidades de gestão de loja e de relatórios — tipicamente reservadas a gerentes e subgerentes — a qualquer colaborador comum apenas com sessão iniciada. Tratando-se de um sistema de gestão de horários de uma loja Levi's, isto poderia permitir a um colaborador sem privilégios visualizar ou, dependendo da extensão futura desses módulos, alterar dados sensíveis de gestão operacional da loja, como relatórios de desempenho ou configurações administrativas. É um exemplo de manual de como a segurança por *routing* é fragiliza quando depende de *strings* de URL duplicadas em vez de uma fonte única de verdade, e por isso destacamos este caso de forma proativa nesta defesa — a transparência sobre o erro faz parte do nosso processo de qualidade.

Pergunta do Professor: E a correção? Garantiram que não volta a acontecer, ou apenas corrigiram a string?

Resposta do Aluno (Tiago/Francisco):

A correção imediata foi o realinhamento dos prefixos comparados com os *paths* reais:

```
if (path.startsWith("/web/gestao-loja")) {
    return webAppService.obterPermissoes(idUtilizador, idLoja)
        .podeGerirLoja();
}
if (path.startsWith("/web/relatorios")) {
    return webAppService.obterPermissoes(idUtilizador, idLoja)
        .podeVerRelatorios();
}
return true;
```

Em conjunto, atualizámos o teste de integração `WebAcessoPerfisIntegrationTest` para usar os novos URLs, garantindo que o caso de teste passa a falhar caso o *path* volte a ficar desalinhado da regra de autorização. Reconhecemos, contudo, que a mitigação estrutural ideal — que identificámos como trabalho futuro — seria eliminar a duplicação de *strings* literais de rota entre o `@RequestMapping` dos controllers e o `WebGuardInterceptor`, centralizando-as em constantes partilhadas, de forma a que uma futura renomeação de rota provoque um erro de compilação em vez de uma falha silenciosa de autorização.

4 Evolução e Schema de Base de Dados

Pergunta do Professor: Vocês usam `spring.jpa.hibernate.ddl-auto=update`. Isso é, em geral, considerado perigoso ou inadequado para produção. Expliquem-me por que escolheram esta configuração e que problema concreto isso vos causou.

Resposta do Aluno (Tiago/Francisco):

Assumimos que `ddl-auto=update` é uma opção adequada para o ritmo de um projeto académico em desenvolvimento ativo, mas reconhecemos as suas limitações: o Hibernate, com esta configuração, consegue **adicionar** colunas novas a tabelas existentes quando deteta uma entidade com um campo novo, mas não garante que essa adição ocorra de forma consistente em todos os ambientes, nem lida bem com alterações de tipo de coluna ou com bases de dados que já tinham sido populadas antes da entidade ser alterada. Foi exatamente isto que aconteceu: a entidade `Turno.java` foi atualizada para incluir o campo `ativo`, mas em ambientes de base de dados já existentes e populados com dados de demonstração, essa coluna não estava garantidamente presente, criando um *drift* entre o modelo de dados em código e o schema real em PostgreSQL — manifestado como falhas em testes de integração que dependiam dessa coluna.

Pergunta do Professor: Mostrem-me exatamente onde, no código Java, esse campo `ativo` está definido, e expliquem-me a diferença entre as duas ocorrências que encontramos — uma em `Turno` e outra em `RegrasLoja`.

Resposta do Aluno (Tiago/Francisco):

Em `API/Modules/Turno.java`, o campo está definido como:

```
@Column(name = "ativo", nullable = false)
private Boolean ativo = true;
```

e serve para marcar um turno-tipo (por exemplo, “Manhã” ou “Noite”) como ativo ou descontinuado, sem necessidade de o eliminar e perder o histórico de horários que já o referenciam por chave estrangeira.

Em `API/Modules/RegrasLoja.java`, o campo é estruturalmente idêntico:

```
@Column(name = "ativo", nullable = false)
private Boolean ativo = true;
```

mas a sua finalidade de negócio é distinta: permite desativar uma regra de negócio específica para uma loja em concreto — por exemplo, uma exceção temporária a uma política de escala — sem apagar o registo, preservando rastreabilidade e auditoria. Ambos os casos seguem o mesmo padrão de *soft-delete* via flag booleana, em vez de eliminação física de linhas, que é a prática recomendada quando outras tabelas têm chaves estrangeiras a apontar para esses registos.

Pergunta do Professor: E como é que resolveram, de forma definitiva, o problema do `ddl-auto=update` não ser fiável para introduzir esta coluna em ambientes já existentes?

Resposta do Aluno (Tiago/Francisco):

Criámos o script `sql/migracao-junho2026.sql`, que aplica explicitamente, via SQL idempotente, todas as alterações de schema necessárias, independentemente do que o Hibernate tenha ou não conseguido sincronizar automaticamente. O passo relevante para a coluna `ativo` em `turnos` é:

```
ALTER TABLE public.turnos
  ADD COLUMN IF NOT EXISTS ativo BOOLEAN NOT NULL DEFAULT TRUE;
```

e o passo equivalente para `regras_loja` é:

```
ALTER TABLE public.regras_loja
  ADD COLUMN IF NOT EXISTS ativo BOOLEAN NOT NULL DEFAULT TRUE;
```

A cláusula `IF NOT EXISTS` garante que o script pode ser executado repetidamente, em qualquer ambiente — de desenvolvimento, de testes ou, hipoteticamente, de produção — sem provocar erro caso a coluna já exista, e sem necessidade de inspecionar manualmente o estado atual de cada base de dados antes de o correr. O script inclui ainda, num passo anterior, um bloco `DO`

\$\$... END \$\$ que verifica condicionalmente, via `information_schema.columns`, se a coluna `turnos.tipo` ainda é do tipo enum antes de a converter para `VARCHAR(50)`, evitando reexecutar uma conversão de tipo já efetuada. Todo o script está encapsulado numa única transação (`BEGIN ... COMMIT`), garantindo atomicidade: ou todas as alterações de schema são aplicadas, ou nenhuma é, prevenindo estados intermédios inconsistentes.

Pergunta do Professor: Porque não confiaram inteiramente no `ddl-auto=update` e deixaram o Hibernate sincronizar tudo sozinho, já que é a opção configurada em `application.properties`?

Resposta do Aluno (Tiago/Francisco):

Porque o `ddl-auto=update` do Hibernate é deliberadamente conservador e aditivo: nunca remove colunas, nunca altera tipos de coluna existentes, e em alguns casos — nomeadamente perante mudanças de tipo, como a nossa conversão de `tipo_turno_enum` para `VARCHAR(50)` — simplesmente não atua, exigindo intervenção manual via SQL. Confiar apenas nesta configuração para um ambiente com dados de demonstração já carregados teria deixado a base de dados num estado parcialmente migrado, sem aviso explícito de falha. Por isso optámos por uma estratégia híbrida e disciplinada: o Hibernate continua a tratar do mapeamento corrente em ambientes novos, mas qualquer alteração estrutural que afete bases de dados já existentes é formalizada e versionada num script SQL explícito, idempotente e auditável, que documenta a própria razão da alteração no seu comentário inicial — inclusive referenciando o commit de código que motivou a mudança, o que consideramos uma boa prática de rastreabilidade entre o histórico do código e o histórico do schema.